

Transformer Encoder-Decoder and Decoder Mathematics

CSE427

- 1 Encoder-decoder flow
- 2 Query, Key, Value meaning
- 3 Self-attention and cross-attention
- 4 Decoder generation process
- 5 Full numerical example

Encoder reads input; decoder generates output.

Why Positional Encoding Is Needed

A Transformer does not read words one by one like an RNN.

For example:

I will go to school

The order is:

I = 0, will = 1, go = 2, to = 3, school = 4

Word embedding tells the model the meaning of the word.

But word embedding alone does not tell the model the position of the word.

So we add positional encoding:

Final input vector = Token embedding + Positional encoding

What Positional Encoding Does

Suppose the word is:

go

Its token embedding may be:

$$E_{go} = [e_1, e_2, e_3, e_4]$$

If “go” is at position 2, then it gets a positional encoding vector:

$$PE_{go} = [p_1, p_2, p_3, p_4]$$

The final input to the Transformer is:

$$X_{go} = E_{go} + PE_{go}$$

$$X_{go} = [e_1 + p_1, e_2 + p_2, e_3 + p_3, e_4 + p_4]$$

Embedding gives meaning; positional encoding gives order.

Sinusoidal Positional Encoding Formula

For token position pos and dimension index i :

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Where:

pos = position of the token

d_{model} = embedding dimension

Even dimensions use sine; odd dimensions use cosine.

Why Sine and Cosine?

Sine and cosine create smooth numerical patterns.

Different positions get different vectors.

For example:

$pos = 0 \rightarrow$ one pattern

$pos = 1 \rightarrow$ another pattern

$pos = 2 \rightarrow$ another pattern

The same formula can be used for any sentence length.

Sinusoidal encoding gives each position a unique numerical signal.

Important idea:

The model can distinguish word order without using recurrence.

Positional Encoding Example: Word “go”

Sentence:

I will go to school

The word “go” has position:

$$pos = 2$$

Suppose the embedding dimension is:

$$d_{model} = 4$$

So we calculate four positional encoding values:

$$PE(2,0), \quad PE(2,1), \quad PE(2,2), \quad PE(2,3)$$

Because $d_{model} = 4$, the positional encoding vector has 4 values.

Calculate $PE(2, 0)$ and $PE(2, 1)$

For $PE(2, 0)$:

$$PE(2, 0) = \sin\left(\frac{2}{10000^{0/4}}\right)$$

$$10000^{0/4} = 10000^0 = 1$$

$$PE(2, 0) = \sin(2)$$

$$PE(2, 0) \approx 0.9093$$

For $PE(2, 1)$:

$$PE(2, 1) = \cos(2)$$

$$PE(2, 1) \approx -0.4161$$

Calculate $PE(2, 2)$ and $PE(2, 3)$

For $PE(2, 2)$:

$$PE(2, 2) = \sin\left(\frac{2}{10000^{2/4}}\right)$$

$$10000^{2/4} = 10000^{1/2} = 100$$

$$PE(2, 2) = \sin\left(\frac{2}{100}\right) = \sin(0.02)$$

$$PE(2, 2) \approx 0.019998$$

For $PE(2, 3)$:

$$PE(2, 3) = \cos(0.02)$$

$$PE(2, 3) \approx 0.9998$$

Final Positional Encoding Vector for "go"

We calculated:

$$PE(2, 0) \approx 0.9093$$

$$PE(2, 1) \approx -0.4161$$

$$PE(2, 2) \approx 0.019998$$

$$PE(2, 3) \approx 0.9998$$

Therefore:

$$PE_{go} = [0.9093, -0.4161, 0.019998, 0.9998]$$

If the token embedding is:

$$E_{go} = [e_1, e_2, e_3, e_4]$$

then final input is:

$$X_{go} = [e_1 + 0.9093, e_2 - 0.4161, e_3 + 0.019998, e_4 + 0.9998]$$

Positional Encoding in the “Lets go” Example

Now consider the example:

Lets go

Token embeddings:

$$\text{Lets} = [1.87, 0.09]$$

$$\text{go} = [-0.78, 0.27]$$

Positional encodings:

$$PE_{\text{Lets}} = [0.0, 1.0]$$

$$PE_{\text{go}} = [-0.9, 0.4]$$

Add positional encoding:

$$\text{Lets} = [1.87, 0.09] + [0.0, 1.0] = [1.87, 1.09]$$

$$\text{go} = [-0.78, 0.27] + [-0.9, 0.4] = [-1.68, 0.67]$$

$[1.87, 1.09]$ and $[-1.68, 0.67]$ are now ready for attention.

Encoder-Decoder Big Picture

Example task:

Input: Lets go

Output: Vamos

The Transformer has two parts:

Encoder

Decoder

- Encoder reads the input sentence.
- Decoder generates output tokens one by one.
- Decoder uses encoder output through cross-attention.

What the Encoder Does

Input sentence:

Lets go

Tokens:

Lets, go

Steps:

- 1 Convert tokens into embeddings.
- 2 Add positional encoding.
- 3 Create Query, Key, Value.
- 4 Apply encoder self-attention.
- 5 Produce encoder output vectors.

Encoder output = contextual memory of input tokens.

What are Query, Key, and Value?

For a token representation X :

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V$$

where W_Q, W_K, W_V are learnable matrices.

Q = Query: what I am looking for

K = Key: what I offer for matching

V = Value: information I pass forward

Encoder Calculation Starts

Input sentence:

Lets go

Token embeddings:

$$\text{Lets} = [1.87, 0.09]$$

$$\text{go} = [-0.78, 0.27]$$

After positional encoding:

$$X_L = [1.87, 1.09]$$

$$X_G = [-1.68, 0.67]$$

Here:

$$L = \text{Lets}, \quad G = \text{go}$$

Now encoder self-attention begins.

$$W_Q = \begin{bmatrix} 1.1 & 0.6 \\ -2.8 & 2.4 \end{bmatrix}$$

$$W_K = \begin{bmatrix} -1.7 & 0.5 \\ -1.5 & 0.9 \end{bmatrix}$$

$$W_V = \begin{bmatrix} 1.5 & -1.0 \\ -0.3 & -0.2 \end{bmatrix}$$

For each encoder token:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Calculate Q_L for “Lets”

$$X_L = [1.87, 1.09]$$

$$Q_L = X_L W_Q$$

$$Q_L = [1.87, 1.09] \begin{bmatrix} 1.1 & 0.6 \\ -2.8 & 2.4 \end{bmatrix}$$

First component:

$$1.87(1.1) + 1.09(-2.8) = 2.057 - 3.052 = -0.995 \approx -1.0$$

Second component:

$$1.87(0.6) + 1.09(2.4) = 1.122 + 2.616 = 3.738 \approx 3.7$$

$$Q_L = [-1.0, 3.7]$$

Calculate K_L and V_L for “Lets”

$$K_L = X_L W_K$$

$$K_L = [1.87, 1.09] \begin{bmatrix} -1.7 & 0.5 \\ -1.5 & 0.9 \end{bmatrix}$$

$$K_L = [-4.7, 1.9]$$

$$V_L = X_L W_V$$

$$V_L = [1.87, 1.09] \begin{bmatrix} 1.5 & -1.0 \\ -0.3 & -0.2 \end{bmatrix}$$

$$V_L = [2.5, -2.1]$$

$$K_L = [-4.7, 1.9], \quad V_L = [2.5, -2.1]$$

Calculate Q_G for “go”

$$X_G = [-1.68, 0.67]$$

$$Q_G = X_G W_Q$$

$$Q_G = [-1.68, 0.67] \begin{bmatrix} 1.1 & 0.6 \\ -2.8 & 2.4 \end{bmatrix}$$

First component:

$$-1.68(1.1) + 0.67(-2.8) = -1.848 - 1.876 = -3.724 \approx -3.7$$

Second component:

$$-1.68(0.6) + 0.67(2.4) = -1.008 + 1.608 = 0.600$$

$$Q_G = [-3.7, 0.6]$$

Calculate K_G and V_G for “go”

$$K_G = X_G W_K$$

$$K_G = [-1.68, 0.67] \begin{bmatrix} -1.7 & 0.5 \\ -1.5 & 0.9 \end{bmatrix}$$

$$K_G = [1.9, -0.2]$$

$$V_G = X_G W_V$$

$$V_G = [-1.68, 0.67] \begin{bmatrix} 1.5 & -1.0 \\ -0.3 & -0.2 \end{bmatrix}$$

$$V_G = [-2.7, 1.5]$$

$$K_G = [1.9, -0.2], \quad V_G = [-2.7, 1.5]$$

Encoder Self-Attention Scores for “Lets”

For “Lets”, use query:

$$Q_L = [-1.0, 3.7]$$

Compare with both keys:

$$K_L = [-4.7, 1.9], \quad K_G = [1.9, -0.2]$$

Score with “Lets”:

$$\begin{aligned} Q_L K_L^T &= (-1.0)(-4.7) + 3.7(1.9) \\ &= 4.7 + 7.03 = 11.73 \approx 11.7 \end{aligned}$$

Score with “go”:

$$\begin{aligned} Q_L K_G^T &= (-1.0)(1.9) + 3.7(-0.2) \\ &= -1.9 - 0.74 = -2.64 \approx -2.6 \end{aligned}$$

$\text{Scores for Lets} = [11.7, -2.6]$

Encoder Self-Attention Scores for “go”

For “go”, use query:

$$Q_G = [-3.7, 0.6]$$

Compare with both keys:

$$K_L = [-4.7, 1.9], \quad K_G = [1.9, -0.2]$$

Score with “Lets”:

$$\begin{aligned} Q_G K_L^T &= (-3.7)(-4.7) + 0.6(1.9) \\ &= 17.39 + 1.14 = 18.53 \approx 18.5 \end{aligned}$$

Score with “go”:

$$\begin{aligned} Q_G K_G^T &= (-3.7)(1.9) + 0.6(-0.2) \\ &= -7.03 - 0.12 = -7.15 \approx -7.2 \end{aligned}$$

$\text{Scores for go} = [18.5, -7.2]$

For "Lets":

$$\text{softmax}(11.7, -2.6) \approx [1.0, 0.0]$$

Meaning:

Lets attends mostly to Lets.

For "go":

$$\text{softmax}(18.5, -7.2) \approx [1.0, 0.0]$$

Meaning:

go also attends mostly to Lets.

Attention weights decide how much value information is used.

Encoder Attention Output

Value vectors:

$$V_L = [2.5, -2.1]$$

$$V_G = [-2.7, 1.5]$$

For "Lets":

$$Attention_L = 1.0V_L + 0.0V_G$$

$$= [2.5, -2.1]$$

For "go":

$$Attention_G = 1.0V_L + 0.0V_G$$

$$= [2.5, -2.1]$$

$$Attention_L = Attention_G = [2.5, -2.1]$$

Encoder Output Update

Original encoder inputs:

$$X_L = [1.87, 1.09]$$

$$X_G = [-1.68, 0.67]$$

Attention outputs:

$$Attention_L = [2.5, -2.1]$$

$$Attention_G = [2.5, -2.1]$$

Update "Lets":

$$\begin{aligned} X'_L &= X_L + Attention_L \\ &= [1.87, 1.09] + [2.5, -2.1] \\ &= [4.37, -1.01] \end{aligned}$$

Update "go":

$$\begin{aligned} X'_G &= X_G + Attention_G \\ &= [-1.68, 0.67] + [2.5, -2.1] \\ &= [0.82, -1.43] \end{aligned}$$

Final Encoder Outputs

After encoder self-attention:

$$\text{Lets} = [4.37, -1.01]$$

$$\text{go} = [0.82, -1.43]$$

These are the encoder memory vectors.

The decoder will use them during cross-attention.

Encoder output becomes K, V source for decoder cross-attention.

What the Decoder Does

The decoder generates output step by step.

$$\langle EOS \rangle \rightarrow \text{Vamos}$$

Then:

$$\langle EOS \rangle, \text{Vamos} \rightarrow \langle EOS \rangle$$

When the decoder produces:

$$\langle EOS \rangle$$

generation stops.

Decoder is autoregressive: previous outputs help predict the next output.

Decoder Has Two Attention Blocks

1. Decoder self-attention

Decoder tokens attend to previous decoder tokens.

2. Encoder-decoder cross-attention

Decoder token attends to encoder outputs.

Self-attention: decoder looks at decoder.

Cross-attention: decoder looks at encoder.

Attention Calculation

Step 1: Compare Query with Keys.

$$\text{score}(Q_i, K_j) = Q_i K_j^T$$

Step 2: Apply softmax.

$$a_{ij} = \frac{e^{\text{score}(Q_i, K_j)}}{\sum_j e^{\text{score}(Q_i, K_j)}}$$

Step 3: Take weighted sum of Values.

$$\text{Attention}_i = \sum_j a_{ij} V_j$$

Scores $\rightarrow$ Softmax weights $\rightarrow$ Weighted Values

Self-Attention vs Cross-Attention

Attention Type	Query Comes From	Key/Value Come From
Encoder self-attention	Encoder tokens	Encoder tokens
Decoder self-attention	Decoder tokens	Decoder tokens
Cross-attention	Decoder tokens	Encoder outputs

In cross-attention, Q is from decoder, K, V are from encoder.

Example Setup

Input sentence:

Lets go

Assume encoder outputs are already computed:

Lets = $[4.37, -1.01]$

go = $[0.82, -1.43]$

These are encoder memory vectors.

The decoder starts with:

$\langle EOS \rangle$

Decoder Start Token

Embedding of $\langle EOS \rangle$:

$$[2.70, -1.34]$$

Positional encoding:

$$[0.0, 1.0]$$

Add them:

$$[2.70, -1.34] + [0.0, 1.0]$$

$$= [2.70, -0.34]$$

So decoder input is:

$$X_E = [2.70, -0.34]$$

$$W_Q = \begin{bmatrix} 0.4 & 0.4 \\ -0.3 & 0.1 \end{bmatrix}$$

$$W_K = \begin{bmatrix} 0.4 & -0.7 \\ -0.4 & 0.3 \end{bmatrix}$$

$$W_V = \begin{bmatrix} -1.1 & -0.7 \\ -0.4 & 1.3 \end{bmatrix}$$

For $X_E = [2.70, -0.34]$:

$$Q_E = X_E W_Q, \quad K_E = X_E W_K, \quad V_E = X_E W_V$$

Calculate Q_E

$$Q_E = [2.70, -0.34] \begin{bmatrix} 0.4 & 0.4 \\ -0.3 & 0.1 \end{bmatrix}$$

First component:

$$2.70(0.4) + (-0.34)(-0.3) = 1.08 + 0.102 = 1.182 \approx 1.2$$

Second component:

$$2.70(0.4) + (-0.34)(0.1) = 1.08 - 0.034 = 1.046 \approx 1.0$$

$$Q_E = [1.2, 1.0]$$

Calculate K_E and V_E

$$K_E = [2.70, -0.34] \begin{bmatrix} 0.4 & -0.7 \\ -0.4 & 0.3 \end{bmatrix}$$

$$K_E = [1.2, -1.8]$$

$$V_E = [2.70, -0.34] \begin{bmatrix} -1.1 & -0.7 \\ -0.4 & 1.3 \end{bmatrix}$$

$$V_E = [-2.8, -2.3]$$

$$K_E = [1.2, -1.8], \quad V_E = [-2.8, -2.3]$$

Decoder Self-Attention Score

Compute dot product:

$$\begin{aligned} Q_E K_E^T &= [1.2, 1.0] \cdot [1.2, -1.8] \\ &= 1.2(1.2) + 1.0(-1.8) \\ &= 1.44 - 1.8 = -0.36 \approx -0.4 \end{aligned}$$

Since there is only one decoder token:

$$\text{softmax}(-0.4) = 1.0$$

$\text{Attention weight} = 1.0$

$$Attention_E = 1.0V_E$$

$$= 1.0[-2.8, -2.3]$$

$$= [-2.8, -2.3]$$

Add to original decoder input:

$$[2.70, -0.34] + [-2.8, -2.3]$$

$$= [-0.1, -2.6]$$

After self-attention: $[-0.1, -2.6]$

Cross-Attention Begins

Decoder representation after self-attention:

$$E = [-0.1, -2.6]$$

Encoder outputs:

$$L = [4.37, -1.01]$$

$$G = [0.82, -1.43]$$

where:

$$L = \text{Lets}, \quad G = \text{go}$$

In cross-attention:

Q from decoder, K, V from encoder

$$W_Q = \begin{bmatrix} 1.5 & -0.3 \\ 0.3 & -1.0 \end{bmatrix}$$

$$W_K = \begin{bmatrix} -1.1 & 0.3 \\ -0.3 & -0.8 \end{bmatrix}$$

$$W_V = \begin{bmatrix} 1.1 & 0.6 \\ -1.2 & -0.5 \end{bmatrix}$$

Cross-Attention Query from Decoder

$$Q_E = EW_Q$$

$$Q_E = [-0.1, -2.6] \begin{bmatrix} 1.5 & -0.3 \\ 0.3 & -1.0 \end{bmatrix}$$

First component:

$$(-0.1)(1.5) + (-2.6)(0.3) = -0.15 - 0.78 = -0.93 \approx -0.9$$

Second component:

$$(-0.1)(-0.3) + (-2.6)(-1.0) = 0.03 + 2.6 = 2.63 \approx 2.6$$

$$Q_E = [-0.9, 2.6]$$

Key and Value for “Lets”

For:

$$L = [4.37, -1.01]$$

$$K_L = LW_K = [-4.5, 2.1]$$

$$V_L = LW_V = [6.0, 3.1]$$

Detailed first component of K_L :

$$4.37(-1.1) + (-1.01)(-0.3)$$

$$= -4.807 + 0.303 = -4.504 \approx -4.5$$

$$K_L = [-4.5, 2.1], \quad V_L = [6.0, 3.1]$$

Key and Value for “go”

For:

$$G = [0.82, -1.43]$$

$$K_G = GW_K = [-0.5, 1.4]$$

$$V_G = GW_V = [2.6, 1.2]$$

Detailed first component of K_G :

$$0.82(-1.1) + (-1.43)(-0.3)$$

$$= -0.902 + 0.429 = -0.473 \approx -0.5$$

$$K_G = [-0.5, 1.4], \quad V_G = [2.6, 1.2]$$

$$Q_E = [-0.9, 2.6]$$

$$K_L = [-4.5, 2.1], \quad K_G = [-0.5, 1.4]$$

Score with Lets:

$$Q_E K_L^T = (-0.9)(-4.5) + 2.6(2.1)$$

$$= 4.05 + 5.46 = 9.51 \approx 9.5$$

Score with go:

$$Q_E K_G^T = (-0.9)(-0.5) + 2.6(1.4)$$

$$= 0.45 + 3.64 = 4.09 \approx 4.1$$

$$\boxed{\text{Scores} = [9.5, 4.1]}$$

Cross-Attention Softmax and Output

$$\text{softmax}(9.5, 4.1) \approx [1.0, 0.0]$$

So the decoder attends mostly to:

Lets

Attention output:

$$1.0V_L + 0.0V_G$$

$$= 1.0[6.0, 3.1] + 0.0[2.6, 1.2]$$

$$= [6.0, 3.1]$$

$\text{Cross-attention output} = [6.0, 3.1]$

Update Decoder Representation

Before cross-attention:

$$[-0.1, -2.6]$$

Cross-attention output:

$$[6.0, 3.1]$$

Add:

$$[-0.1, -2.6] + [6.0, 3.1]$$

$$= [5.9, 0.5]$$

$$\text{Final decoder vector} = [5.9, 0.5]$$

This vector goes to dense layer and softmax.

Generate First Output Token

Final decoder vector:

[5.9, 0.5]

Dense layer produces scores for vocabulary:

ir, Vamos, y, $\langle EOS \rangle$

Softmax converts scores into probabilities.

If highest probability is:

Vamos

then:

$\langle EOS \rangle \rightarrow \text{Vamos}$

Second Decoder Step

Now decoder input contains:

$\langle EOS \rangle, \text{ Vamos}$

Representation of “Vamos”:

$[-0.86, 2.37]$

Now decoder self-attention happens again.
This time “Vamos” can attend to:

$\langle EOS \rangle$

and:

Vamos

Self-Attention for “Vamos”

Given:

$$K_E = [1.2, -1.8], \quad V_E = [-2.8, -2.3]$$

$$Q_V = [-1.3, -0.1], \quad V_V = [0.0, 3.7]$$

Scores:

$$Q_V K_E^T = -1.1$$

$$Q_V K_V^T = 1.4$$

$$\text{softmax}(-1.1, 1.4) = [0.1, 0.9]$$

Vamos attends 10% to EOS and 90% to itself.

Update “Vamos” After Self-Attention

Attention output:

$$\begin{aligned} & 0.1V_E + 0.9V_V \\ &= 0.1[-2.8, -2.3] + 0.9[0.0, 3.7] \\ &= [-0.28, -0.23] + [0.0, 3.33] \\ &= [-0.28, 3.10] \end{aligned}$$

Add to original Vamos:

$$\begin{aligned} & [-0.86, 2.37] + [-0.28, 3.10] \\ &= [-1.14, 5.47] \approx [-1.2, 5.5] \end{aligned}$$

After self-attention: $[-1.2, 5.5]$

Cross-Attention for “Vamos”

Decoder query:

$$Q_V = [-0.1, -5.1]$$

Encoder keys and values:

$$K_L = [-4.5, 2.1], \quad V_L = [6.0, 3.1]$$

$$K_G = [-0.5, 1.4], \quad V_G = [2.6, 1.2]$$

Scores:

$$Q_V K_L^T = -10.3$$

$$Q_V K_G^T = -7.1$$

$$\text{softmax}(-10.3, -7.1) = [0.0, 1.0]$$

So “Vamos” attends mostly to:

go

Update “Vamos” After Cross-Attention

Cross-attention output:

$$0.0V_L + 1.0V_G$$

$$= [2.6, 1.2]$$

Before cross-attention:

$$[-1.2, 5.5]$$

Add:

$$[-1.2, 5.5] + [2.6, 1.2]$$

$$= [1.4, 6.7]$$

Final Vamos representation = $[1.4, 6.7]$

Generate Next Token

Final vector:

[1.4, 6.7]

Dense layer and softmax produce probabilities over:

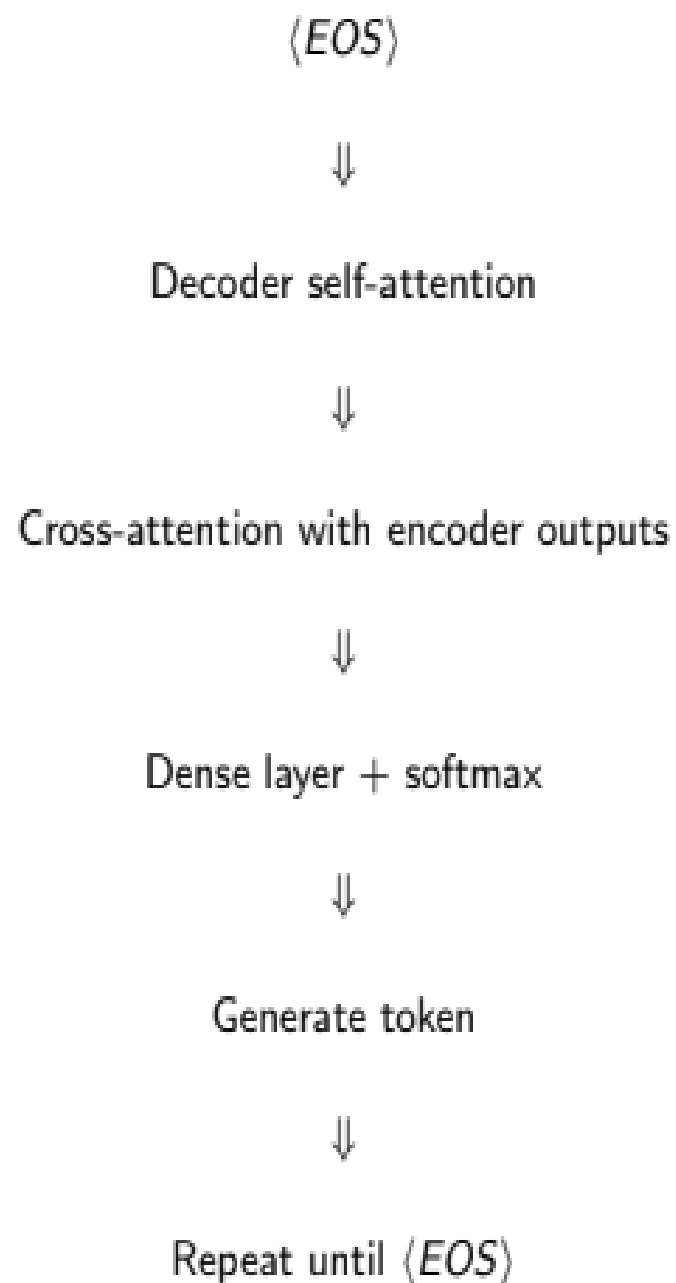
ir, Vamos, y, $\langle EOS \rangle$

If highest probability is:

$\langle EOS \rangle$

then decoding stops.

Generated sequence: Vamos



- Encoder reads the input sentence.
- Decoder generates output tokens one by one.
- Query is compared with Keys.
- Softmax produces attention weights.
- Values are combined using attention weights.
- Decoder self-attention looks at decoder tokens.
- Cross-attention looks at encoder outputs.
- Dense layer and softmax generate the next token.

Decoder = self-attention + cross-attention + token generation